

14.1 Retrieval-Augmented Generation

CS224N L10: retrieve then generate. Cosine 1 / 0.707 / 0. Not Week 10 RAFT.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

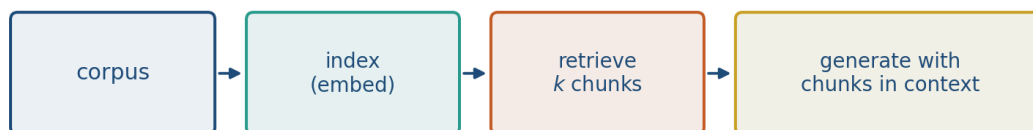
RAG is retrieve, then generate. The language model does not have to store every fact in its weights. You give it **chunks** at request time and ask it to answer from those chunks.

Stanford **CS224N 2026 L10**: this is **non-parametric memory at inference**. Note **10.3 RAFT** is a **training** recipe that teaches the model to use retrieved docs. This week is the retrieve-then-generate **stack**. Do not mix the names.

1. Three stages

1. **Index**. Split documents, embed each chunk, store vectors plus the raw text and an id.
2. **Retrieve**. Embed the query, take top- k by cosine (or a hybrid with keywords).
3. **Generate**. Stuff the chunks into a prompt: answer the question, cite ids, say when the context is missing.

RAG: non-parametric memory at inference (and, in RAFT, at train time)



The generator is still a next-token model. RAG changes its **conditioning**, not its architecture. A wrong index cannot be saved by a prettier prompt. Dense embeddings (Week 1, Week 5) are one index; keyword overlap is another. Lab 14 uses counts so the ranking is something you can print.

Karpathy's browser demo is the same loop with the web as the index: emit a search, read hits, then write. Custom GPTs that file your PDFs are the same loop with a private index.

2. What can fail

Retrieval misses the supporting chunk (recall@k). Retrieval returns a near-miss that the model then over-trusts. The model **ignores** a good chunk and hallucinates. Citations that do not match the used

sentences are the RAG version of unfaithful CoT (note **13.3**).

Chunk size, overlap, embedding choice, and k are method. Lab 14 uses bag-of-words cosine so you can see the ranking without a GPU. ReAct (note **14.2**) can treat retrieve as one tool among others, including a calculator.

A no-retrieval baseline is the parametric model on the same questions. If RAG does not beat it, the index or the prompt is the story, not “we added RAG.”

3. Measurement

Report retrieval recall@k on a labeled set of (question, gold chunk ids), then answer accuracy with an attribution check: does the cited id actually contain the claim? Always include a no-retrieval baseline. “We used RAG” is not a result.

4. Teaching this note

About **35 minutes** at the board, then **~10 minutes** of video. Lecture ideas follow **CS224N 2026 L10**.

- **0–12 min.** Index / retrieve / generate. Cosine as the ranker. What lives in the index besides the vector.
 - **12–22 min.** Failure modes: miss, near-miss, ignore, unfaithful citation.
 - **22–33 min.** Worked cosine on three toy docs.
 - **Then** play Karpathy intro **27:43–33:32** (tool use: browser as retrieve-then-read) and mention **40:45–42:15** (custom GPTs / files as a private index). Pause when the model emits a search and reads the hits.
-

5. Worked example

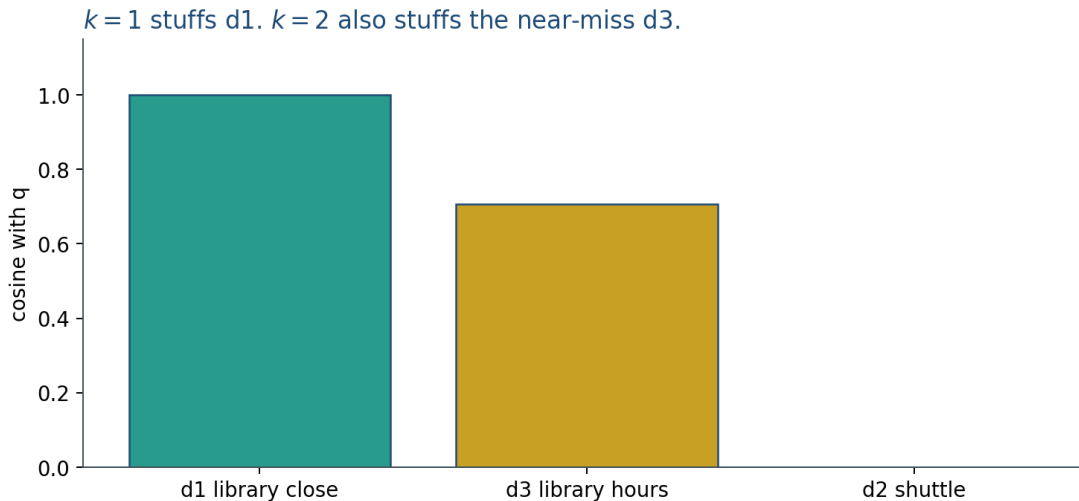
Three chunks, vocab {library, close, shuttle}, bag-of-words counts:

id	text (toy)	vector
d1	library close	(1, 1, 0)
d2	shuttle loop	(0, 0, 1)
d3	library hours	(1, 0, 0)

Query q = “library close”, vector (1, 1, 0). Cosine $\cos(a, b) = a^T b / (\|a\| \|b\|)$.

- $\cos(q, d1) = 2 / (\sqrt{2} \sqrt{2}) = 1$
- $\cos(q, d2) = 0$
- $\cos(q, d3) = 1 / (\sqrt{2} \cdot 1) \approx 0.707$

Rank: d1, d3, d2. For $k = 1$ you stuff only d1 (right chunk for a closing-time question). For $k = 2$ you also stuff d3: recall can rise, but the generator now sees a near-miss about “hours” that may not include 23:00. Raising k is not free.



Lost in the middle: if the gold chunk sits in the middle of a long stuffed list, models often ignore it. That is a retrieval-plus-position bug, not “the model cannot read.” Keep k small in Lab 14.

If the model then cites [d2] while using the 23:00 fact from d1, that is unfaithful attribution: same bug class as CoT steps that do not match the box.

6. Where students get stuck

- Measuring only answer accuracy. You need recall@ k **and** a citation check.
- Stuffing $k = 20$ and wondering why the model ignores the gold chunk.
- Putting only vectors in the index and dropping the raw text / id, so you cannot cite.

7. Video

Watch [Andrej Karpathy, Intro to Large Language Models](#), second half: **27:43–33:32** (browser / retrieval) and **40:45–42:15** (customization with files).

Pause on emit-search → read hits → write with sources. Same URL as notes **13.1**, **14.2**, and **15.1**. This note is the retrieve-then-generate picture; **14.2** is the tool loop.

8. Practice

1. Why can raising k hurt the answer even if it raises recall?
2. You retrieve the right chunk and the answer is still wrong. Name two different bugs.

3. What belongs in the index besides the embedding vector?
4. Using the three vectors above, compute $\cos(q, d3)$ exactly as a fraction, and write the $k = 2$ id list.
5. Gold chunk is d1. Retrieved top-2 are d3, d2. Recall@2 for this item? Answer cites [d1] . Attribution pass or fail?